DAMG 7370

Designing Advanced Data Architectures
for Business Intelligence

Final Team Project

Food Establishment Inspections — Chicago & Dallas

Gaurav Dalal · Michael Dereje · Pratik Kanade
Northeastern University
Spring 2026

1. Introduction

This report documents the design and implementation of a data engineering pipeline for analyzing food establishment inspection data from two U.S. cities: Chicago, Illinois and Dallas, Texas. The project was built on the Databricks platform following the Medallion Architecture pattern (Bronze → Silver → Gold), with each layer progressively refining the data from its raw source form into a clean, analytics-ready dimensional model.

Chicago data is sourced from the City of Chicago Data Portal and covers inspections from January 2010 through April 2026 (308,431 records, 17 columns). Dallas data is sourced from the City of Dallas Open Data portal and spans October 2016 through February 2024 (78,984 records, 114 columns). The two datasets differ significantly in schema structure: Chicago stores all violations for an inspection in a single pipe-delimited text column, while Dallas uses a wide/denormalized format with 25 repeating violation column groups.

The pipeline implements data profiling using both manual PySpark analysis and the Databricks DQX Profiler, Silver layer validation with 12 cleansing rules, a star schema dimensional model with six dimensions and one fact table, SCD Type 2 tracking on the violation dimension, and Power BI dashboards for inspection analysis. The entire pipeline is orchestrated through a Databricks Job with parallel task execution.

2. Architecture Overview

The pipeline is structured around the Medallion Architecture, which organizes data into three progressively refined layers. Each layer has a distinct storage format, write mode, and purpose.

Layer	Storage	Write Mode	Purpose
Bronze	Unity Catalog Delta	Overwrite	Raw CSV data with cleaned column names. No transformations.
Silver	Unity Catalog Delta	Overwrite	Validated, cleansed, and normalized data. Violations parsed into rows.
Gold	Unity Catalog Delta	Overwrite / Merge	Star schema with surrogate keys, SCD Type 2, and dimension lookups.

Data flows through the pipeline as: Source CSV → Bronze (Delta) → Silver (Delta) → Gold (Delta). The Bronze layer reads CSV files from a Unity Catalog Volume and writes Delta tables with cleaned column names. The Silver layer applies validation rules, parses violations, and derives scores. The Gold layer builds the dimensional model and implements SCD Type 2.

All tables are registered under the workspace.default schema in Unity Catalog, providing governance, lineage tracking, and discoverability across all layers.

3. Environment Setup

3.1 Volume and Data Upload

The source CSV files are stored in a Unity Catalog Volume created at workspace.default.food_inspections. The catalog_setup notebook creates this volume using CREATE VOLUME IF NOT EXISTS. Both CSV files (chicago_inspection_dataset.csv and dallas_inspection_dataset.csv) were uploaded to this volume through the Databricks UI.

```
05:26 PM (27s)                                    1: Create volume if not exist
# Create volume
spark.sql("CREATE VOLUME IF NOT EXISTS workspace.default.food_inspections")
See performance (1)
DataFrame[]
```

3.2 Bronze Ingestion

The bronze_ingestion notebook reads both CSV files with multiLine and escape options to handle embedded quotes and newlines. A clean_column_names() function uses regex to convert all column names to lowercase_underscore format, which is required for Delta Lake compatibility. The cleaned DataFrames are written as Delta tables in overwrite mode.

Table	Rows	Columns
bronze_chicago_inspections	308,431	17
bronze_dallas_inspections	78,984	114

Dallas has 114 columns because it stores violations in 25 repeating groups of 4 columns each (description, points, detail, memo), plus 14 core columns. Chicago stores violations as a single text column.

```
Chicago Bronze: 308431 rows, 17 cols
Dallas Bronze: 78984 rows, 114 cols

--- Chicago columns ---
  inspection_id
  dba_name
  aka_name
  license
  facility_type
  risk
  address
  city
  state
  zip
  inspection_date
  inspection_type
  results
  violations
  latitude
  longitude
  location
```

4. Data Profiling

Profiling was performed in two stages: manual PySpark analysis and automated DQX profiling. Both ran against the Bronze tables to assess structure, quality, and characteristics before any transformations.

4.1 Manual Profiling

The manual profiling notebook analyzes null counts per column, value distributions for categorical fields, violation structure, and data range checks. Key findings:

Chicago: 28.05% of inspections had no violations (86,503 of 308,431). Top inspection types were Canvass (160,034), License (41,130), and Canvass Re-Inspection (34,376). Results included Pass, Pass w/ Conditions, Fail, No Entry, and several others.

```
=== CHICAGO VIOLATIONS OVERVIEW ===
Total: 308431 | With: 221928 (71.95%) | Without: 86503 (28.05%)

=== CHICAGO: VIOLATIONS PER INSPECTION ===
```

Table

	v_count	count
1	1	30047
2	2	36037
3	3	35995
4	4	31144
5	5	24481
6	6	18411
7	7	13198
8	8	9515
9	9	6707
10	10	4673
11	11	3302
12	12	2404
13	13	1617
14	14	1198
15	15	886

35 rows | 12.27s runtime

Refreshed 2 hours ago

Dallas: Only 11 null restaurant names. Score range was -26 to 100 (mean 90.86, median 92). Six records had negative scores. 259 records had ZIP+4 format codes (9–10 characters). Violation slot usage tapered from 91.67% at slot 1 to under 1% at slot 25.

```
=== DALLAS NEGATIVE SCORES ===
Count: 6
+-------------------------------------+---------------+----------------+
|restaurant_name                      |inspection_date|inspection_score|
+-------------------------------------+---------------+----------------+
|MI PUEBLITO TAQUERIA Y PA            |2023-06-18     |-26             |
|KFC                                  |2023-06-23     |-13             |
|THE LAST STAND                       |2023-09-13     |-15             |
|MACARON                              |2023-09-15     |-3              |
|2AT&T FOOD HALL - COMMON AREA (LVL 01)|2023-09-15     |-5              |
+-------------------------------------+---------------+----------------+
only showing top 5 rows

=== DALLAS ZIP CODE FORMAT CHECK ===
+----------+-----+
|zip_length|count|
+----------+-----+
|         5|78725|
|         9|   67|
|        10|  192|
+----------+-----+
```

```
=== CHICAGO ZIP FORMAT CHECK ===
+----------+------+
|zip_length| count|
+----------+------+
|         5|308389|
+----------+------+
```

4.2 DQX Profiler

The Databricks DQX Profiler was run on both Bronze tables using the databricks-labs-dqx library. It auto-generated 27 rules for Chicago covering is_not_null checks, min_max bounds on inspection_id and zip, and is_in constraints on categorical fields like risk, state, and city. Dallas profiling was run on core columns only (excluding the 100 violation columns) to keep output manageable. DQX findings confirmed the manual profiling results and established the validation expectations used in the Silver layer.

=== CHICAGO COLUMN STATISTICS ===

Table

	column	count	count_null	count_non_null	min
1	inspection_id	1000	0	1000	2629909
2	dba_name	1000	0	1000	#1 CHOP SUEY RESTAURANT, INC
3	aka_name	1000	2	998	#1 CHOP SUEY
4	license	1000	0	1000	0
5	facility_type	1000	21	979	Bakery
6	risk	1000	1	999	All
7	address	1000	0	1000	1 E JACKSON BLVD
8	city	1000	1	999	BERWYN
9	state	1000	0	1000	CA
10	zip	1000	0	1000	60402
11	inspection_type	1000	0	1000	Canvass
12	results	1000	0	1000	Fail
13	violations	1000	325	675	1. PERSON IN CHARGE PRESENT, DEMONSTRATES KNOWLEDGE, AND PERFORMS DUTIES - Comments: 2-101.11 OBSERVED NO PERS...
14	latitude	1000	3	997	41.67226959743
15	longitude	1000	3	997	-87.906874134

17 rows | 4.13s runtime

Refreshed 2 hours ago

=== CHICAGO DQX RULES ===

Table

	column	rule	description	parameters
1	inspection_id	is_not_null		
2	inspection_id	min_max	Real min/max values were used	{'min': 2629909, 'max': 2633683}
3	dba_name	is_not_null		
4	aka_name	is_not_null	Column aka_name has 0.2% of null values (allowed 1.0%)	
5	license	is_not_null		
6	facility_type	is_not_null_or_empty		{'trim_strings': True}
7	risk	is_not_null	Column risk has 0.1% of null values (allowed 1.0%)	
8	risk	is_in		{'in': ['Risk 1 (High)', 'Risk 2 (Medium)', 'Risk 3 (Lo
9	address	is_not_null		
10	city	is_not_null	Column city has 0.1% of null values (allowed 1.0%)	
11	city	is_in		{'in': ['CHICAGO', 'Chicago', 'BERWYN', 'TORRAN
12	state	is_not_null		
13	state	is_in		{'in': ['IL', 'CA']}
14	zip	is_not_null		
15	zip	min_max	Real min value was used. Max was capped by 3 sigmas. avg=60658.093, stddev=944.8628875277238, max=90501	{'min': 60402, 'max': 63493}

27 rows | 4.13s runtime

Refreshed 2 hours ago

5. Silver Layer Implementation

The Silver layer applies validation rules, parses violations into individual rows, derives scores, and produces a unified schema for both cities. Each city has its own Silver notebook due to the fundamentally different source structures.

5.1 Validation Rules

The following rules are applied as expectations. Rows that fail any rule are dropped from the Silver output.

#	Rule	Applies To	Action	Rows Affected
1	Restaurant/DBA Name not null	Both	Drop	11 (Dallas)
2	Inspection Date not null	Both	Drop	0
3	Inspection Type not null	Both	Drop	0
4	Zip code valid 5-digit format	Both	Drop/Truncate	259 truncated, 43 dropped
5	Score cannot exceed 100	Dallas	Drop	0
6	Results cannot be null	Chicago	Drop	Part of 43 total
7	Deduplicate violations per inspection	Both	Deduplicate	62,305 (Chi), 1,086 (Dal)
8	Score ≥ 90 with > 3 violations	Dallas	Drop inspection	18,314 flagged
9	No PASS with Urgent/Critical	Dallas	Drop inspection	1 (Uptown Urban Market)
10	Derive scores from results	Chicago	Transform	All Chicago rows
11	Negative scores invalid	Dallas	Drop	6

5.2 Chicago Silver

After applying Rules 1–6, 43 rows were dropped from the Chicago Bronze table (308,431 → 308,388). Inspection scores were derived from the results text: Pass = 90, Pass w/ Conditions = 80, Fail = 70, No Entry = 0, all others = NULL.

The violations column contains pipe-delimited text with the format: "CODE. DESCRIPTION - Comments: NOTES | CODE. DESC - Comments: NOTES". This was parsed by splitting on "|", exploding into individual rows, and using regex to extract the violation code (leading digits), description (text before "- Comments:"), and comments (text after "- Comments:"). This produced 1,002,162 violation rows. After deduplication on (inspection_id, violation_code, violation_description), 62,305 duplicates were removed, leaving 939,857 violation rows.

Violation severity was classified from the violation text: keywords like urgent, critical, priority, or imminent mapped to Critical; serious or major mapped to Serious; everything else was Minor.

Inspections with no violations (86,472 rows) were retained with NULL violation fields and unioned with the violation rows to produce the final Chicago Silver table of 1,026,329 rows. These no-violation records are essential for accurate pass-rate calculations.

```
Inspections with violations: 221916
Inspections without violations: 86472
Total violation rows after parsing: 1002162

=== SAMPLE PARSED VIOLATIONS ===
```

Table

	inspection_id	violation_code	violation_description	violation_comments
1	2633665	10	ADEQUATE HANDWASHING SINKS PROPERLY SUPPLIED AND ACCESSIBLE	OBSERVED HAND WASHING ONLY SIGNAGE NEED FOR THE ONE C
2	2633665	38	INSECTS, RODENTS, & ANIMALS NOT PRESENT	OBSERVED A GAP AT THE BOTTOM LEFT OF THE SINGLE EXIT DOOI
3	2633665	51	PLUMBING INSTALLED; PROPER BACKFLOW DEVICES	OBSERVED A LEAK AT THE RIGHT FAUCET OF THE FOUR COMPARTI
4	2633638	38	INSECTS, RODENTS, & ANIMALS NOT PRESENT	OBSERVED OVER 20 RAT DROPPINGS SCATTERED ON FLOOR UNDE
5	2633638	51	PLUMBING INSTALLED; PROPER BACKFLOW DEVICES	5-205.15 OBSERVED LEAK UNDER UTILITY SINK AND HANDWASHI
6	2633638	55	PHYSICAL FACILITIES INSTALLED, MAINTAINED & CLEAN	6-501.114 OBSERVED UNUSED EQUIPMENT AND CRATES STORED (
7	2633654	36	THERMOMETERS PROVIDED & ACCURATE	OBSERVED BROKEN THERMOMETERS INSIDE UPRIGHT REFRIGERAT
8	2633654	47	FOOD & NON-FOOD CONTACT SURFACES CLEANABLE, PROPERLY DESIGNED, CONSTRUCTED & USED	OBSERVED RUSTED SHELVING RACKS INSIDE UPRIGHT COCA COLA
9	2633654	53	TOILET FACILITIES: PROPERLY CONSTRUCTED, SUPPLIED, & CLEANED	OBSERVED GARBAGE RECEPTACLE IN WOMENS TOILETROOM WITHOI
10	2633654	55	PHYSICAL FACILITIES INSTALLED, MAINTAINED & CLEAN	OBSERVED FLOORS IN REAR PREP AREA AROUND ALL HEAVY COO

10 rows | 2.96s runtime — Refreshed 2 hours ago

```
=== NULL CHECK ON REQUIRED FIELDS ===
  dba_name: 0 nulls ✓
  inspection_date: 0 nulls ✓
  inspection_type: 0 nulls ✓
  zip_code: 0 nulls ✓
  results: 0 nulls ✓

=== SCORE DISTRIBUTION ===
```

Table

	results	inspection_score	count
1	Business Not Locat...	null	92
2	Fail	70	346816
3	No Entry	0	17000

7 rows | 7.47s runtime — Refreshed 2 hours ago

```
=== SEVERITY DISTRIBUTION ===
```

Table

	violation_severity	count
1	Minor	747711
2	Critical	152917
3	null	86472

4 rows | 7.47s runtime — Refreshed 2 hours ago

5.3 Dallas Silver

After applying validation Rules 1–5 and 11, the Dallas dataset was cleaned of null restaurant names, negative scores, and invalid ZIP formats. ZIP+4 codes were truncated to 5 digits.

Dallas stores violations in a wide format with 25 repeating column groups. These were unpivoted into rows using a SQL stack() expression that converted 25 groups of (description, points, detail, memo) columns into individual rows, dropping empty slots. This produced the initial violation rows.

Rule 8 flagged 18,314 inspections with score ≥ 90 and more than 3 violations. Rule 9 flagged 1 inspection (Uptown Urban Market, score 94 with a Critical HACCP violation). After deducting overlaps, 18,269 distinct bad inspections were removed. The remaining rows were deduplicated (1,086 duplicates removed), producing the final Dallas Silver table of 306,446 rows.

Dallas has no native inspection ID. A synthetic ID was generated via SHA2(restaurant_name + inspection_date), prefixed with "DAL-". Latitude and longitude were parsed from a combined field using regex.

```
violation_counts: pyspark.sql.connect.dataframe.DataFrame = [restaurant_name: string, inspection_date: date ... 3 more fields]
rule8_violations: pyspark.sql.connect.dataframe.DataFrame = [restaurant_name: string, inspection_date: date ... 3 more fields]
rule9_violations: pyspark.sql.connect.dataframe.DataFrame = [restaurant_name: string, inspection_date: date ... 3 more fields]
bad_inspections: pyspark.sql.connect.dataframe.DataFrame = [restaurant_name: string, inspection_date: date]
Rule 8 — Score >= 90 with > 3 violations: 18314
Rule 9 — Score >= 90 with Urgent/Critical: 1
Total bad inspections to drop: 18269
```

```
=== SEVERITY DISTRIBUTION ===
```

Table

	violation_severity	count
1	Minor	133055
2	Critical	88497
3	Serious	84894

3 rows | 10.31s runtime

Refreshed 2 hours ago

```
=== RULE 8 VERIFICATION ===
Violations of Rule 8 remaining: 0 ✓
```

6. Gold Layer — Dimensional Model

The Gold layer transforms the cleaned Silver tables into a star schema. The fact table grain is one row per violation per inspection, with foreign keys to six dimension tables.

6.1 Dimension Tables

Table	Rows	Natural Key	SCD Type	Description
dim_date	4,551	date_key (YYYYMMDD)	Type 1	Calendar dimension with year, quarter, month, day, fiscal year, weekend flag
dim_establishment	58,270	license (Chi) / name|address (Dal)	Type 1	Business info: name, AKA, license, facility type, risk level
dim_location	27,904	address|zip_code	Type 1	Geographic info: address, city, state, zip, latitude, longitude
dim_violation	950 (initial)	violation_code + source_city	Type 2	Violation codes and descriptions from both cities
dim_inspection_type	113	inspection_type + source_city	Type 1	Inspection type lookup with categories (Routine, Re-Inspection, Complaint, Other)
dim_inspection_result	7	result_text	Type 1	Result text, derived score, result category (Pass/Fail/Other)

All Type 1 dimensions are written in overwrite mode with surrogate keys generated using monotonically_increasing_id(). Keys are internally consistent per run.

6.2 Fact Table

Column	Type	Description
inspection_violation_id	Surrogate key	Unique row identifier
date_key	FK → dim_date	Inspection date
establishment_key	FK → dim_establishment	Business/restaurant
location_key	FK → dim_location	Geographic location
violation_key	FK → dim_violation	Violation code (NULL for no-violation inspections)
inspection_type_key	FK → dim_inspection_type	Type of inspection
result_key	FK → dim_inspection_result	Inspection result
source_city	Degenerate dim	Chicago or Dallas
inspection_id	Degenerate dim	Original inspection identifier
inspection_score	Measure	Numeric score (derived for Chicago, native for Dallas)
violation_points	Measure	Points assigned to violation (Dallas only)
violation_slot	Degenerate dim	Violation position 1–25 (Dallas only)

The fact table contains 1,332,775 rows (1,026,329 from Chicago + 306,446 from Dallas). The FK null check shows 0 nulls for date_key, location_key, inspection_type_key, and result_key. The 86,500 null violation_key rows correspond to Chicago inspections with no violations. The 62 null establishment_key rows are edge cases where Chicago license numbers did not match the dimension.

```
fact_inspection_violation: 1332775 rows
dim_date: 4551 rows
dim_establishment: 58270 rows
dim_location: 27904 rows
dim_violation: 1898 rows
dim_inspection_type: 113 rows
dim_inspection_result: 7 rows

=== SCD2 STATUS ===
Current records: 950
Historical records: 948

=== FACT TABLE FK NULL CHECK ===
  date_key: 0 nulls ✓
  establishment_key: 62 nulls ⚠ has nulls
  location_key: 0 nulls ✓
  violation_key: 86500 nulls ⚠ has nulls
  inspection_type_key: 0 nulls ✓
  result_key: 0 nulls ✓

=== FACT BY SOURCE CITY ===
```

Table +

	source_city	count
1	Chicago	1026329
2	Dallas	306446

7. SCD Type 2 — dim_violation

The dim_violation table implements SCD Type 2 to track changes to violation descriptions over time. This was implemented using manual PySpark logic rather than Delta Live Tables, providing full control over the merge process.

7.1 Table Structure

The SCD2 columns are: violation_key (surrogate), violation_code, source_city, violation_description, violation_detail, violation_severity, is_current (boolean), effective_date, and end_date (NULL for current records).

7.2 Initial Load

The first cell in the Gold notebook unions all distinct violations from both Silver tables (deduped on violation_code + source_city + violation_description) and inserts them as current records with is_current = true, effective_date = 2010-01-01, and end_date = NULL. This produced 950 records.

```
Changed violations detected: 114
Expired: 948 | New current: 948

dim_violation total: 1898
Current: 950
Historical: 948

=== VIOLATION CODE 38 CHICAGO (SCD2 proof) ===
```

Table

	violation_key	violation_code	source_city	violation_description	violation_detail
1	58	38	Chicago	VENTILATION: ROOMS AND EQUIPMENT VENTED AS REQUIRED: PLUMBING: INSTALLED AND MAINTAINED	MINOR LEAK NOTED AT BOTTOM OF 3-(
2	105	38	Chicago		
3	5	38	Chicago	INSECTS, RODENTS, & ANIMALS NOT PRESENT	6-202.15 OBSERVED 1/4" GAP AT BOT
4	1032	38	Chicago		
5	1031	38	Chicago	INSECTS, RODENTS, & ANIMALS NOT PRESENT	6-202.15 OBSERVED 1/4" GAP AT BOT
6	1039	38	Chicago	VENTILATION: ROOMS AND EQUIPMENT VENTED AS REQUIRED: PLUMBING: INSTALLED AND MAINTAINED	MINOR LEAK NOTED AT BOTTOM OF 3-(

6 rows | 16.67s runtime

Refreshed 2 hours ago

7.3 Merge Logic

The second cell compares incoming Silver violations against current dim_violation records. It joins on (violation_code, source_city) and identifies rows where violation_description has changed. For each changed violation: the existing current record is expired (is_current set to false, end_date set to the current date), and a new current record is inserted with the updated description.

On our pipeline run, the merge detected 114 changed violation keys, which produced 948 expired records and 948 new current records. The final dim_violation table contains 1,898 rows: 950 current and 948 historical. The high number of expired records relative to the 114 changed keys is because multiple description variants exist per violation code, and the join on (code, city) cross-matches all variants.

The SCD2 behavior was verified by querying violation code 38 for Chicago, which showed both historical (expired) and current records with different descriptions and the correct effective/end dates.

```
Changed violations detected: 114
Expired: 948 | New current: 948

dim_violation total: 1898
Current: 950
Historical: 948

=== VIOLATION CODE 38 CHICAGO (SCD2 proof) ===
```

Table

	violation_key	violation_code	source_city	violation_description	violation_detail
1	58	38	Chicago	VENTILATION: ROOMS AND EQUIPMENT VENTED AS REQUIRED: PLUMBING: INSTALLED AND MAINTAINED	MINOR LEAK NOTED AT BOTTOM OF 3-(
2	105	38	Chicago		
3	5	38	Chicago	INSECTS, RODENTS, & ANIMALS NOT PRESENT	6-202.15 OBSERVED 1/4" GAP AT BOT
4	1032	38	Chicago		
5	1031	38	Chicago	INSECTS, RODENTS, & ANIMALS NOT PRESENT	6-202.15 OBSERVED 1/4" GAP AT BOT
6	1039	38	Chicago	VENTILATION: ROOMS AND EQUIPMENT VENTED AS REQUIRED: PLUMBING: INSTALLED AND MAINTAINED	MINOR LEAK NOTED AT BOTTOM OF 3-(

6 rows | 16.67s runtime

Refreshed 2 hours ago

8. Job Orchestration

The pipeline is orchestrated using a Databricks Job defined in YAML (job-manual-gdalal.yaml). The job contains three tasks: bronze_to_silver_chicago and bronze_to_silver_dallas run in parallel with no dependencies, followed by the gold task which depends on both Silver tasks completing. The Bronze ingestion step runs separately as a one-time prerequisite. The YAML configuration specifies WORKSPACE as the notebook source with performance-optimized compute and queue enabled.

9. Power BI Dashboards

The BI layer connects Power BI to the Databricks SQL Warehouse using a Personal Access Token. All Gold layer tables are loaded for visualization. The dashboard includes three pages: an Inspection Overview with KPI cards and trend charts, a Violations Analysis page with top violations and severity breakdowns, and a detailed Inspection Report page with searchable filters and a location map. The Power BI file (inspections_for_chicago_and_dallas.pbix) and connection file (databricks-Serverless Starter Warehouse.pbids) are included in the repository.

10. Testing and Verification

10.1 Silver Layer Validation

After running both Silver notebooks, the validation summary confirmed zero nulls on all required fields (dba_name, inspection_date, inspection_type, zip_code, results for Chicago; dba_name, inspection_date, inspection_type, zip_code for Dallas). The Rule 8 verification check confirmed zero remaining violations (score ≥ 90 with > 3 violations) in the Dallas Silver table.

10.2 Gold Layer Verification

The Gold validation cell checks every foreign key column in the fact table for null values. Results: date_key (0 nulls), establishment_key (62 nulls — edge cases), location_key (0 nulls), violation_key (86,500 nulls — expected for no-violation inspections), inspection_type_key (0 nulls), result_key (0 nulls). The fact table row count (1,332,775) matches the sum of both Silver tables exactly.

10.3 SCD Type 2 Verification

The SCD2 merge was verified using SQL queries in the scd2_checks notebook. A test record was inserted into Bronze Chicago with a modified description for violation code 38. After re-running the Silver and Gold notebooks, the dim_violation table correctly showed both the historical (expired) and current versions of the violation with the appropriate effective_date and end_date values.

10.4 Row Count Summary

Stage	Chicago	Dallas	Combined
Bronze	308,431	78,984	387,415
Silver (after all rules)	1,026,329	306,446	1,332,775
Gold Fact	1,026,329	306,446	1,332,775